

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA



DOCUMENTAZIONE MANUTENZIONE

ANNO ACCADEMICO 2025/2026

[REPOSITORY GITHUB](#)

Giuseppe Martusciello NF22500109
Giuseppe Sindoni NF22500108

Contents

1	Introduzione	5
1.1	Contesto e Motivazioni	5
1.2	Dominio Applicativo	5
1.3	Architettura del Sistema e Tecnologie Utilizzate	6
1.4	Sicurezza e Protezione dei Dati	6
1.5	Obiettivo del Progetto	6
2	Inquadramento del Reverse Engineering	7
2.1	Specificità del Caso di Studio	7
2.2	Approccio adottato	7
2.3	Knowledge Transfer	8
2.3.1	Attività svolte	8
3	Artefatti Documentali e Validazione Modello–Implementazione	9
3.1	Artefatti Documentali Disponibili	9
3.2	Inclusione dei Diagrammi	9
3.3	Metodo di Verifica della Coerenza	10
3.3.1	Analisi Funzionale (Use Case Diagram)	10
3.3.2	Analisi Strutturale (Class Diagram)	10
3.3.3	Analisi dei Flussi Operativi (Use Case Cockburn)	11
3.4	Esito della Validazione	11
4	Osservazioni Emersa dalla Revisione Architetturale	12
4.1	Premessa	12
4.2	Vincoli Progettuali Identificati	12
5	Software Configuration Management	13
5.1	Repository e Branching Strategy	13
5.2	Gestione delle Pull Request	13
5.3	Gestione delle Change Request	13

5.4	Conclusione	14
6	Change Request #1: Introduzione autenticazione multi-factor	15
6.1	Problema identificato	15
6.2	Obbiettivo della modifica	15
6.3	Analisi preliminare dell'impatto	16
6.3.1	Impatto sui requisiti	16
6.4	Impatto sulla struttura del codice	16
6.4.1	Starting Impact Set (SIS)	16
6.4.2	Actual Impact Set (AIS)	17
6.4.3	Differenza tra SIS e AIS	19
6.5	Stakeholder coinvolti	19
6.6	Strategia di Implementazione	19
6.6.1	Piano di implementazione	20
6.7	Tecnologie e strumenti	23
6.7.1	Backend	23
6.7.2	Frontend	23
6.8	Gestione delle dipendenze e compatibilità	23
6.8.1	Dipendenze interne	23
6.8.2	Retro-compatibilità	23
6.8.3	Gestione della sessione	23
6.9	Aggiornamento della documentazione	24
6.9.1	Use case	24
6.9.2	Class Diagram	25
6.9.3	Sequence diagram flusso di Login	25
6.10	Conclusioni	25
6.10.1	Raccomandazioni future	26
7	Change Request #2: Creazione prenotazioni da parte del medico	27
7.1	Problema identificato	27
7.2	Obiettivo della modifica	27
7.3	Analisi preliminare dell'impatto	28
7.3.1	Impatto sui requisiti	28
7.4	Impatto sulla struttura del codice	28
7.4.1	Starting Impact Set (SIS)	28
7.4.2	Actual Impact Set (AIS)	29

7.4.3	Differenza tra SIS e AIS	30
7.5	Stakeholder coinvolti	30
7.6	Strategia di Implementazione	30
7.6.1	Piano di implementazione	31
7.7	Tecnologie e strumenti	34
7.7.1	Backend	34
7.7.2	Frontend	34
7.8	Gestione delle dipendenze e compatibilità	34
7.8.1	Dipendenze interne	34
7.8.2	Retro-compatibilità	34
7.9	Aggiornamento della documentazione	35
7.10	Use Case	35
7.11	Sequence Diagram	36
7.12	Conclusioni	36
8	Change Request #3: Messa in sicurezza dell'onboarding dei pazienti	37
8.1	Problema identificato	37
8.2	Obiettivo della modifica	37
8.3	Analisi preliminare dell'impatto	38
8.3.1	Impatto sui requisiti	38
8.4	Impatto sulla struttura del codice	38
8.4.1	Starting Impact Set (SIS)	38
8.4.2	Actual Impact Set (AIS)	39
8.4.3	Differenza tra SIS e AIS	41
8.5	Stakeholder coinvolti	41
8.6	Strategia di Implementazione	42
8.6.1	Revisione del Modello Dati e del Flusso di Onboarding	42
8.6.2	Migrazione dei dati legacy (Invite → User)	42
8.6.3	Implementazione della Logica di Onboarding nel Backend	44
8.6.4	Rimozione del Flusso Invite-Based e Hardening delle API	44
8.6.5	Integrazione Frontend	45
8.7	Tecnologie e strumenti	45
8.7.1	Backend	45
8.7.2	Frontend	45
8.8	Gestione delle dipendenze e compatibilità	46
8.8.1	Dipendenze interne	46

8.8.2	Retro-compatibilità	46
8.8.3	Gestione del primo accesso	46
8.9	Aggiornamento della documentazione	46
8.9.1	Use Case	46
8.10	Conclusioni	48
8.10.1	Raccomandazioni future	48

1 Introduzione

1.1 Contesto e Motivazioni

Il progetto **SymbioCare** nasce dall'esigenza di rispondere ai bisogni della medicina sportiva moderna, integrando una piattaforma digitale che consenta la gestione, il monitoraggio e l'analisi continua dei dati clinici e biometrici dei pazienti.

Nel contesto attuale, i professionisti del settore non si limitano più a erogare visite cliniche standard, ma necessitano di strumenti avanzati che offrano un supporto completo nella cura dei pazienti. **SymbioCare** si propone come risposta a questa necessità, evolvendo oltre la gestione tradizionale delle visite e introducendo un approccio proattivo, basato sul monitoraggio costante dei parametri vitali.

La piattaforma mira a ottimizzare vari aspetti della gestione sanitaria, tra cui:

- Monitoraggio continuo dei parametri vitali;
- Gestione centralizzata delle prenotazioni;
- Tracciabilità storica dei dati medici;

In questo modo, **SymbioCare** offre un approccio innovativo, incentrato sulla prevenzione e sul miglioramento della qualità della cura, riducendo al contempo gli oneri burocratici per i professionisti e aumentando l'accessibilità e la sicurezza per i pazienti.

1.2 Dominio Applicativo

Il sistema è stato progettato per soddisfare i requisiti specifici del dominio della **medicina sportiva**, un settore che richiede alte performance in termini di affidabilità, sicurezza e gestione dei dati. Le caratteristiche distintive del dominio sono:

- Elevata affidabilità dei dati biometrici e clinici;
- Integrazione con dispositivi biometrici esterni per il rilevamento in tempo reale dei parametri.

SymbioCare si colloca così nell'ambito delle piattaforme di *Digital Health*, portando avanti la visione di un sistema sanitario interconnesso, proattivo e a supporto del benessere continuo dei pazienti.

1.3 Architettura del Sistema e Tecnologie Utilizzate

SymbioCare è strutturato come una piattaforma integrata e modulare, che include i seguenti componenti principali:

- **Backend:** API RESTful sviluppate con *NestJS*, con gestione dei dati tramite *PostgreSQL* e *TypeORM*;
- **Frontend Web:** Interfaccia per i medici sviluppata con *React* e *Vite*, per una UI reattiva e interattiva;
- **Frontend Mobile:** App per i pazienti in fase di sviluppo, costruita con *Kotlin* e *Jetpack Compose*;
- **Database:** *PostgreSQL*, un database relazionale robusto, per la gestione sicura dei dati clinici.

1.4 Sicurezza e Protezione dei Dati

La protezione dei dati sanitari e la sicurezza del sistema sono elementi fondamentali di **SymbioCare**, considerando la sensibilità delle informazioni gestite. Le principali misure adottate per garantire la sicurezza includono:

- **Autenticazione sicura:** sistema basato su JWT (JSON Web Tokens), con utilizzo di *access token* e *refresh token*, memorizzati in *cookie HttpOnly* per proteggere i dati sensibili;
- **Hashing delle password:** utilizzo di *Bcrypt* per l'hashing sicuro delle credenziali utente, riducendo il rischio di compromissioni;
- **Separazione dei ruoli:** differenziazione tra medici e pazienti, con accesso limitato alle informazioni in base al ruolo, per garantire la riservatezza e il controllo sui dati.

1.5 Obiettivo del Progetto

Il progetto **SymbioCare** si prefigge l'obiettivo di migliorare l'efficienza del lavoro medico sportivo e di garantire un monitoraggio continuo e sicuro dei parametri biometrici. Attraverso una piattaforma digitale avanzata, il sistema non solo ottimizza la gestione dei pazienti e degli appuntamenti, ma promuove anche la salute proattiva attraverso il monitoraggio in tempo reale dei parametri vitali. Il risultato finale è un sistema integrato che facilita il lavoro dei professionisti medici e migliora l'esperienza del paziente, mantenendo sempre alta la sicurezza e la qualità dei dati trattati.

2 Inquadramento del Reverse Engineering

2.1 Specificità del Caso di Studio

Nel presente progetto, il sistema analizzato:

- È stato sviluppato nell'ambito di una tesi di laurea triennale
- È estremamente recente
- Non ha subito modifiche successive alla discussione della tesi
- Possiede documentazione completa e dettagliata

La documentazione è stata redatta contestualmente allo sviluppo dell'implementazione e rappresenta fedelmente il sistema attuale.

2.2 Approccio adottato

Alla luce di tali caratteristiche, non si è reso necessario un processo di Reverse Engineering ricostruttivo, inteso come ricostruzione di modelli a partire dal codice. Sebbene il sistema non presenti caratteristiche tipiche di un sistema legacy, l'introduzione di Change Request in un contesto di manutenzione evolutiva richiede comunque una fase iniziale di analisi e validazione.

Pertanto, l'attività svolta ha avuto lo scopo di:

- Validare formalmente la coerenza tra diagrammi e codice
- Garantire una conoscenza condivisa del sistema
- Identificare eventuali vincoli architetturali in vista dell'evoluzione

2.3 Knowledge Transfer

Poiché uno dei membri del team è l'autore originale del sistema, è stata pianificata una fase strutturata di Knowledge Transfer al fine di condividere la conoscenza architetturale e ridurre la dipendenza da conoscenza implicita.

2.3.1 Attività svolte

Durante la fase di Knowledge Transfer sono stati discussi:

- Flussi principali del dominio applicativo
- Modellazione dei ruoli (Doctor, Patient)
- Logica di gestione delle prenotazioni
- Meccanismo di inserimento pazienti tramite invito
- Struttura del database e relazioni tra entità
- Scelte implementative originarie e relative motivazioni

3 Artefatti Documentali e Validazione Modello–Implementazione

3.1 Artefatti Documentali Disponibili

Il sistema oggetto di analisi dispone dei seguenti artefatti documentali, redatti contestualmente allo sviluppo dell’implementazione:

- Use Case Diagram
- Class Diagram
- Tre Use Case dettagliati secondo il template di Cockburn

Tali documenti sono stati prodotti nell’ambito della tesi di laurea triennale e non risultano obsoleti, in quanto il sistema non ha subito modifiche successive alla discussione della tesi.

3.2 Inclusione dei Diagrammi

Al fine di garantire completezza documentale e trasparenza metodologica, si riportano di seguito i riferimenti ai diagrammi originali presenti nella repository ufficiale del progetto, utilizzati come riferimento durante la fase di validazione.

- Use Case Diagram
- Class Diagram (UML)

3.3 Metodo di Verifica della Coerenza

La validazione tra modello e implementazione è stata condotta secondo un approccio sistematico articolato in tre fasi:

3.3.1 Analisi Funzionale (Use Case Diagram)

Per ciascun caso d'uso modellato è stata verificata:

- La presenza dell'endpoint o servizio corrispondente nel backend
- La corretta gestione degli attori previsti
- La coerenza tra dipendenze modellate (include/extend) e flussi reali

La verifica è stata effettuata mediante analisi del codice dei controller e dei service layer, confrontando le responsabilità implementate con quelle descritte nel modello.

3.3.2 Analisi Strutturale (Class Diagram)

La validazione del diagramma delle classi è stata effettuata confrontando :

- Le entità UML con le entità del dominio implementate
- Gli attributi modellati con i campi effettivi delle classi
- Le relazioni UML con le relazioni ORM/database
- Le cardinalità con i vincoli implementativi
- il diagramma E/R fornito da Supabase

L'analisi è stata svolta esaminando:

- Struttura delle entità
- Mapping verso il database
- Dipendenze tra moduli

3.3.3 Analisi dei Flussi Operativi (Use Case Cockburn)

Per i casi d'uso dettagliati è stato effettuato un confronto puntuale tra:

- Precondizioni dichiarate
- Flusso principale
- Flussi alternativi
- Gestione degli errori

La verifica è stata condotta analizzando il comportamento del sistema e il codice responsabile della gestione delle operazioni descritte.

3.4 Esito della Validazione

La fase di verifica ha confermato la piena coerenza tra documentazione e implementazione.

Non sono emerse discrepanze funzionali o strutturali. Il modello architetturale rispecchia fedelmente il sistema attuale.

La documentazione può pertanto essere considerata:

- Aggiornata
- Consistente
- Affidabile come base per la manutenzione evolutiva

4 Osservazioni Emerse dalla Revisione Architettuale

4.1 Premessa

La fase di Knowledge Transfer e di validazione modello-implementazione non si è limitata a una verifica di coerenza formale, ma durante la discussione delle scelte implementative originarie sono emerse alcune considerazioni relative a:

- Evolvibilità del sistema
- Ridondanza informativa
- Vincoli architettureali impliciti
- Flussi operativi fortemente accoppiati

4.2 Vincoli Progettuali Identificati

L'analisi ha evidenziato la presenza di alcune scelte progettuali coerenti con il contesto iniziale del sistema ma potenzialmente limitanti in un'ottica di evoluzione futura.

In particolare, sono stati individuati:

- Vincoli funzionali che influenzano la flessibilità dei flussi operativi
- Accoppiamenti tra entità che aumentano la complessità di gestione
- Meccanismi implementativi che introducono ridondanza strutturale

Le osservazioni emerse non compromettono la correttezza o la coerenza del sistema, ma rappresentano aree di possibile miglioramento in vista dell'introduzione di nuove Change Request.

5 Software Configuration Management

5.1 Repository e Branching Strategy

Il repository del progetto utilizza Git come sistema di versionamento, adottando la strategia di branching GitFlow. Questa strategia prevede i seguenti branch principali:

- **main**: Contiene solo il codice stabile e pronto per il deploy.
- **develop**: Utilizzato per integrare le funzionalità completate e testate. È il punto di partenza per tutte le attività di sviluppo.

Branch secondari per attività specifiche includono:

- **feature/***: Per lo sviluppo di nuove funzionalità.
- **bugfix/***: Per la correzione di difetti.
- **hotfix/***: Per correzioni urgenti direttamente sul branch **main**.

5.2 Gestione delle Pull Request

Le modifiche al codice devono essere integrate tramite Pull Request (PR). Ogni PR deve rispettare le seguenti regole:

- Descrivere la modifica effettuata.
- Fare riferimento alla Change Request associata.
- Superare i controlli automatici (build e test).

5.3 Gestione delle Change Request

Le Change Request (CR) sono il cuore del processo di gestione delle modifiche. Ogni CR segue il ciclo di vita seguente:

1. **Identificazione della CR:** Descrizione del problema o della nuova funzionalità.
2. **Creazione della branch:** Una branch dedicata viene creata da `develop` con il nome conforme allo standard.
3. **Sviluppo e commit:** Implementazione delle modifiche, con commit frequenti e significativi.
4. **Apertura della Pull Request:** La PR viene aperta verso `develop` per la revisione e i test.
5. **Merge:** Una volta approvata, la PR viene unita al branch `develop`.
6. **(Opzionale) Release / Hotfix:** Se necessario, la modifica viene rilasciata o applicata come hotfix a `main`.

5.4 Conclusione

L'adozione di una gestione del codice chiara e strutturata tramite GitFlow e le regole per commit, branch e pull request permette di garantire la tracciabilità delle modifiche, migliorando la qualità del software e facilitando la manutenzione continua.

6 Change Request #1: Introduzione autenticazione multi-factor

6.1 Problema identificato

L'analisi dello stato attuale del sistema ha evidenziato una criticità riconducibile al modello di autenticazione adottato nella versione baseline.

Il sistema implementa un meccanismo di autenticazione basato esclusivamente su credenziali statiche (email e password), con generazione di token JWT e gestione delle sessioni tramite cookie `HttpOnly`. Sebbene tale approccio sia funzionalmente corretto e conforme alle pratiche comuni di autenticazione, esso non risulta pienamente adeguato rispetto al dominio applicativo di riferimento.

In presenza di un eventuale furto di credenziali, un attaccante potrebbe accedere al sistema impersonando legittimamente l'utente, senza ulteriori barriere di sicurezza.

Pertanto, la criticità identificata è riconducibile a un livello di protezione dell'accesso non proporzionato alla sensibilità delle informazioni gestite dal sistema.

6.2 Obiettivo della modifica

La presente Change Request ha l'obiettivo di rafforzare il modello di autenticazione introducendo un secondo fattore di verifica basato su codici temporanei OTP (One-Time Password) generati tramite applicazioni di autenticazione (es. Google Authenticator, Microsoft Authenticator).

L'obiettivo specifico è:

- Consentire agli utenti di abilitare volontariamente un meccanismo di autenticazione a due fattori (2FA)
- Introdurre un passaggio intermedio di verifica nel processo di login
- Ridurre il rischio di accessi non autorizzati in caso di compromissione delle credenziali primarie

6.3 Analisi preliminare dell'impatto

6.3.1 Impatto sui requisiti

La modifica proposta avrà un impatto diretto sui requisiti del sistema, in particolare riguardo alla sicurezza e alla gestione delle credenziali. L'introduzione di un sistema di autenticazione a due fattori (2FA) modifica il requisito di accesso sicuro, che dovrà essere rivisitato per riflettere l'integrazione del secondo fattore di autenticazione.

I requisiti legati alla gestione dei dati sensibili, come quelli clinici, saranno rafforzati dal nuovo flusso di autenticazione, assicurando che le credenziali non siano sufficienti per l'accesso ai dati. Tuttavia, non vi sono modifiche ai requisiti funzionali principali, poiché la modifica riguarda esclusivamente il processo di autenticazione e non la gestione dei dati o le funzionalità esistenti del sistema.

6.4 Impatto sulla struttura del codice

6.4.1 Starting Impact Set (SIS)

In questo caso, l'insieme di impatto previsto per l'introduzione del 2FA include:

Backend

- Aggiunta di una logica per generare e inviare il codice di autenticazione via email o tramite un'app.
- Integrazione di una libreria per la gestione del 2FA: Si prevede l'integrazione di una libreria come `otplib` per la gestione del TOTP (Time based One Time password).
- Modifica del processo di login per includere la validazione del secondo fattore, richiedendo l'otp dopo l'inserimento delle credenziali.
- Aggiornamento dei test di autenticazione.

Frontend

- Implementazione di una schermata aggiuntiva che richiede un codice temporaneo inviato tramite email o un'app di autenticazione (come Google Authenticator).
- Modifica del flusso di login per richiedere il secondo fattore di autenticazione dopo l'inserimento delle credenziali (login/password).

- Gestione della sessione utente aggiornata per consentire l'autenticazione tramite il 2FA, assicurando che l'utente non possa essere autenticato senza aver passato entrambe le fasi di verifica (login e OTP).

Database

- Modifica della tabella **User**: Verranno aggiunti nuovi campi alla tabella User per gestire il nuovo stato dell'autenticazione, come `twoFactorEnabled` e `twoFactorSecret`.

6.4.2 Actual Impact Set (AIS)

L'insieme di impatto effettivo si estende a:

Backend

- Creazione di un nuovo service responsabile della logica TOTP.
- Modifica del service di autenticazione: flusso di login, aggiunta di metodi per la creazione di un nuovo segreto per il 2fa, per la verifica, creazione o conferma di un codice OTP.
- Modifica del controller di autenticazione: aggiunti endpoint necessari per le nuove funzionalita' di 2fa, modificato l'endpoint di signin adattandosi in caso la 2fa sia attiva o meno per l'utente in questione.
- Modificato il modulo di autenticazione aggiornando i nuovi componenti del modulo e providers.
- Introdotte nuove dipendenze esterne: `otplib` per la gestione del TOTP, `qrcode` per la generazione dei codici QR e `uuid` per la generazione dei challenge ID univoci.

Frontend

- Aggiunta di nuove chiamate API per il 2FA: Sono state aggiunte 4 nuove chiamate API per interagire con i nuovi endpoint backend relativi al 2FA, inclusi quelli per la configurazione, conferma e verifica del codice OTP.
- Aggiunta di custom hook per la gestione del 2FA: Sono stati aggiunti 4 nuovi hook per gestire le mutazioni relative al 2FA (configurazione, verifica, conferma e disabilitazione del 2FA).
- Modificati i custom hook di signin e register: sono stati modificati per includere la gestione del flusso di 2FA, adattandosi al fatto che alcuni utenti potrebbero avere la 2FA attivata.

- Creazione di due nuove schermate: una per la verifica del codice OTP durante il login e una per la gestione delle impostazioni utente, tra cui la configurazione e disabilitazione del 2FA.
- Creazione di un componente per l'inserimento del codice OTP, utilizzato durante il flusso di login, e un altro per la gestione del 2FA nelle impostazioni.

Database

- Modifica della tabella **User** mostrate nella tabella 6.1.

Table 6.1: Configurazione Two-Factor Authentication (2FA)

Colonna	Tipo	Nullable	Default	Descrizione
twoFactorEnabled	boolean	No	false	Indica se la 2FA è attiva per l'utente.
twoFactorSecret	varchar	Sì	null	Segreto TOTP attivo, utilizzato per la verifica degli OTP.
twoFactorSecretPending	varchar	Sì	null	Segreto TOTP temporaneo generato durante la configurazione, non ancora confermato.

- Creazione della tabella **Auth_challenge** come descritto nella tabella 6.2

Table 6.2: Schema della Tabella Challenge 2FA

Campo	Tipo	Descrizione
id	uuid (PK)	Identificatore univoco dell'istanza di challenge.
challengeId	varchar (UNIQUE)	Token opaco generato per identificare la sessione di verifica 2FA.
userId	varchar	Riferimento all'utente che ha iniziato il login.
type	varchar	Tipo di challenge (attualmente "LOGIN_2FA").
expiresAt	Date	Timestamp di scadenza (5 minuti dalla creazione).
attempts	integer	Contatore tentativi falliti.
maxAttempts	integer	Soglia massima di tentativi (default: 5).
createdAt	Date (auto)	Timestamp di creazione.

6.4.3 Differenza tra SIS e AIS

Backend

- La creazione di un nuovo service per la logica TOTP e la modifica approfondita del service di autenticazione per la gestione di OTP sono andati oltre le previsioni iniziali.
- Il controller di autenticazione è stato modificato più ampiamente, con l'aggiunta di nuovi endpoint e la gestione del flusso 2FA in base alla configurazione dell'utente.

Frontend

- Sono stati aggiunti nuovi hook e chiamate API per gestire i nuovi flussi legati al 2FA, con un aumento significativo della complessità rispetto alle previsioni iniziali.

Database

- La creazione della tabella **Auth_challenge** per gestire il processo di verifica 2FA rappresenta una differenza significativa rispetto alle previsioni iniziali, che si limitavano alla modifica della tabella User.

6.5 Stakeholder coinvolti

Gli impatti sugli stakeholder sono principalmente legati agli utenti finali (pazienti e medici), che dovranno adattarsi a una nuova modalità di accesso al sistema. In particolare:

- **Patient:** l'introduzione del 2FA potrebbe causare una leggera frustrazione iniziale a causa della necessità di gestire un secondo fattore. Tuttavia, il miglioramento della sicurezza garantirà una protezione maggiore dei dati personali e clinici.
- **Doctor:** gli utenti professionisti potrebbero apprezzare il maggiore livello di sicurezza, sebbene la modifica del processo di accesso possa introdurre un piccolo rallentamento nelle operazioni quotidiane, soprattutto in caso di malfunzionamento temporaneo del sistema OTP.

6.6 Strategia di Implementazione

La soluzione implementata distingue tre macro-scenari:

1. **Setup 2FA** per utenti autenticati (generazione segreto, QR code, conferma OTP).
2. **Disable 2FA** per utenti autenticati (conferma OTP).
3. **Login con 2FA attiva** tramite challenge temporaneo e verifica OTP prima dell'emissione dei token di sessione.

6.6.1 Piano di implementazione

Il piano è organizzato in fasi sequenziali. Ogni fase produce un incremento testabile e riduce il rischio di regressioni.

Fase 1 — Adeguamento Persistenza e Modello Dati

- Estensione della `User` con i campi necessari alla gestione del 2FA (`twoFactorEnabled`, `twoFactorSecret`, `twoFactorSecretPending`).
- Introduzione della tabella `Auth_challenge` per gestire le sessioni temporanee di verifica 2FA durante il login (scadenza, numero tentativi).

Risultato atteso: disponibilità del supporto dati per distinguere utenti con 2FA attiva e gestire il “second step” in modo stateful e sicuro.

Fase 2 — Implementazione Logica 2FA nel Backend

- Introduzione di un service dedicato (*TwoFactorService*) per:
 - generazione segreto TOTP e URL `otpauth://`;
 - verifica OTP con finestra temporale di tolleranza.
- Estensione del service di autenticazione per:
 - login condizionale: emissione token **oppure** generazione challenge se 2FA attiva;
 - gestione setup/confirm/disable della 2FA;
 - gestione tentativi, scadenza e invalidazione della challenge.

Risultato atteso: backend in grado di supportare i due macro-scenari (setup e login con challenge).

Fase 3 — Estensione API e Contratti

- Introduzione di endpoint dedicati per setup, conferma, disabilitazione e verifica OTP in login.
- Adeguamento del contratto dell'endpoint di `signin`: ritorno di una risposta alternativa in caso di 2FA attiva (presenza di `requires2fa` e `challengeId`).

Risultato atteso: API stabili e consumabili dal frontend, con gestione esplicita del caso “richiede 2FA”.

Fase 4 — Integrazione Frontend

- Aggiornamento del layer API per includere le nuove chiamate verso gli endpoint 2FA.
- Introduzione di hook di mutazione (TanStack Query) per orchestrare i flussi 2FA (setup, confirm, verify, disable).
- Estensione del flusso di login:
 - se `requires2fa = true` → redirect a `/2fa` con `challengeId`;
 - altrimenti → flusso di login invariato.
- Introduzione di nuove UI:
 - Pagina di verifica OTP in fase di `(/2fa)` 6.1.

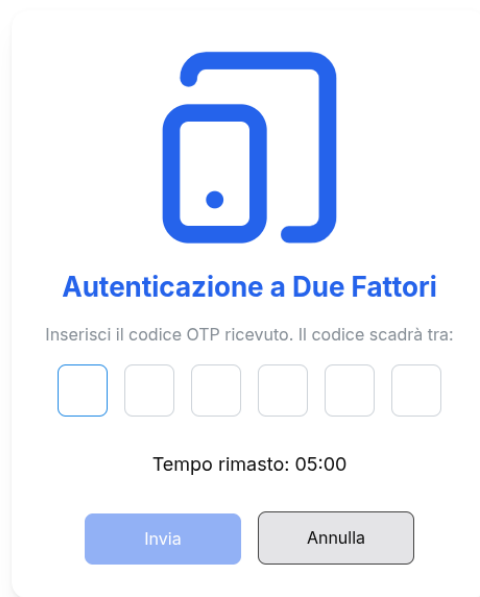


Figure 6.1: Inserimento codice OTP - signin con 2fa abilitata

- pagina impostazioni (`/settings`) con sezione dedicata al 2FA (abilita/disabilita).

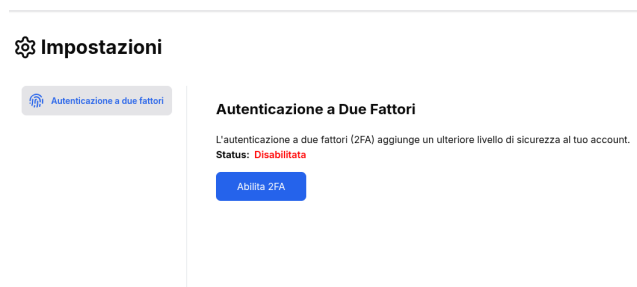


Figure 6.2: Schermata di verifica OTP durante il login.



Figure 6.3: Schermata per l'abilitazione del 2FA.

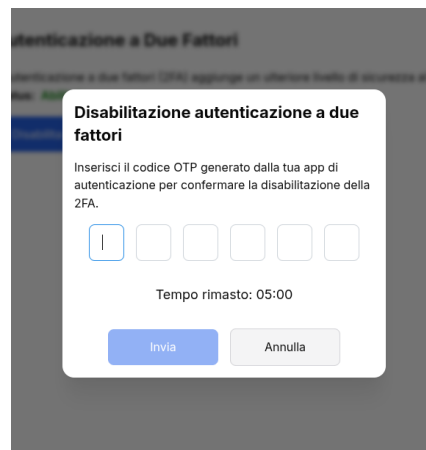


Figure 6.4: Schermata per la disabilitazione del 2FA.

Risultato atteso: esperienza utente completa (setup e login) con navigazione e stati coerenti.

6.7 Tecnologie e strumenti

6.7.1 Backend

- **otplib** per generazione/verifica TOTP.
- **qrcode** per generare il QR code (PNG/Base64) da `otpauth://`.
- **uuid** per generare identificativi di challenge.

6.7.2 Frontend

- **TanStack Query** per gestione asincrona delle mutazioni (`setup/verify/confirm/disable`).
- **Routing** con introduzione delle nuove route (`/2fa`, `/settings`).

6.8 Gestione delle dipendenze e compatibilità

6.8.1 Dipendenze interne

La Change Request introduce dipendenze esplicite tra:

- **AuthService** e **TwoFactorService** (verifica OTP, generazione segreti);
- **AuthService** e **Auth_challenge** (challenge stateful: scadenza e tentativi);
- **Frontend login flow** e **nuova risposta di signin** (caso `requires2fa`).

6.8.2 Retro-compatibilità

Il comportamento precedente è mantenuto:

- utenti senza 2FA attiva completano il login nel modo tradizionale;
- i nuovi campi `twoFactorSecret` e `twoFactorSecretPending` sono `nullable` per non forzare migrazioni logiche sugli utenti pre-esistenti.

6.8.3 Gestione della sessione

La sessione applicativa (cookie JWT/refresh) viene emessa **solo dopo** la verifica OTP per utenti con 2FA attiva. Per evitare sessioni “parziali”, il sistema utilizza una **challenge temporanea** con scadenza e limite tentativi, separata dalla sessione principale.

6.9 Aggiornamento della documentazione

Riguardo la documentazione saranno aggiornati i seguenti diagrammi.

6.9.1 Use case

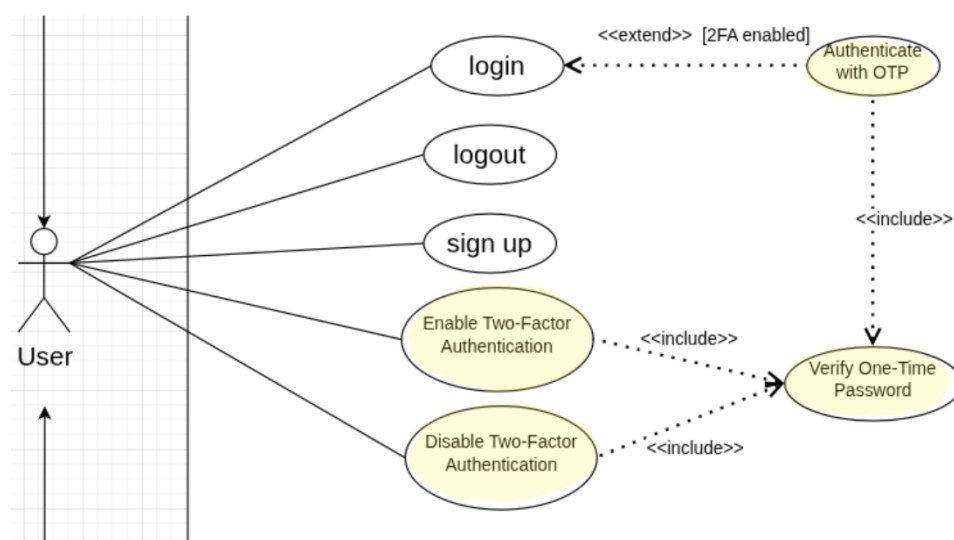


Figure 6.5: Use Case

Come si vede in figura 6.5 sono stati aggiunti 4 nuovi use case:

- abilitazione autenticazione a due fattori
- disabilitazione autenticazione a due fattori
- autenticazione tramite OTP
- verifica tramite OTP

6.9.2 Class Diagram

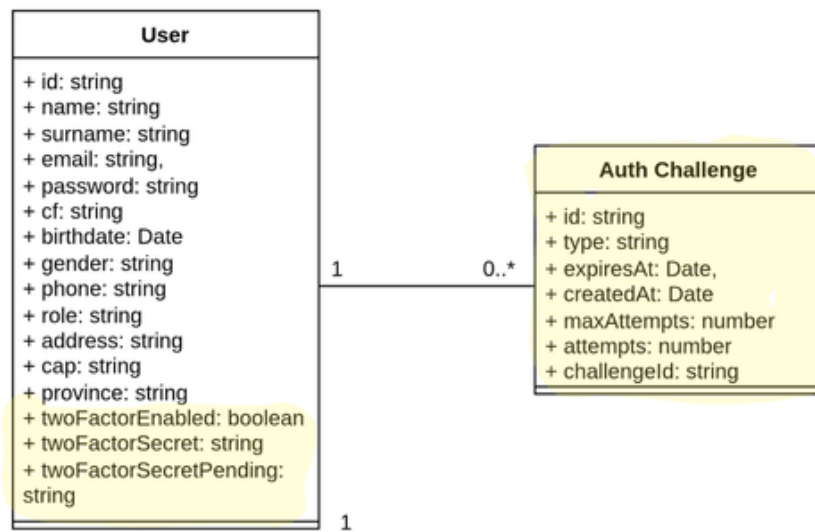


Figure 6.6: Class Diagram

Come mostrato in figura 6.6 sono state inseriti gli attributi necessari alla tabella **User**, aggiunta la tabella **AuthChallenge** e descritta la relazione tra le due tabelle.

6.9.3 Sequence diagram flusso di Login

Nel seguente diagramma viene riportato il flusso aggiornato del signin dopo l'implementazione della change request:

- Diagramma di Sequenza 2FA (HTML)

6.10 Conclusioni

L'introduzione dell'autenticazione a due fattori ha consentito di migliorare in modo significativo il livello di sicurezza del sistema, mitigando il rischio di accessi non autorizzati derivanti dalla compromissione delle credenziali utente.

La modifica ha richiesto interventi distribuiti su più livelli dell'architettura, includendo l'estensione del modello dati, l'introduzione di nuovi componenti applicativi per la gestione dei codici OTP e l'adattamento del flusso di autenticazione tra backend e frontend.

L'analisi dell'impatto ha evidenziato come l'insieme di impatto effettivo sia risultato più ampio rispetto a quello inizialmente previsto, principalmente a causa della necessità di gestire uno stato temporaneo di autenticazione tramite challenge dedicate.

Nonostante l'aumento della complessità del sistema, la soluzione mantiene la retro-compatibilità con gli utenti che non hanno abilitato il 2FA, garantendo continuità operativa e una transizione graduale verso un modello di sicurezza più robusto.

6.10.1 Raccomandazioni future

Come possibile evoluzione del sistema, si suggerisce l'introduzione di meccanismi di recupero dell'account basati su codici di backup, al fine di gestire in modo sicuro la perdita del dispositivo utilizzato per l'autenticazione.

Ulteriori miglioramenti potrebbero includere l'introduzione di sistemi di rate limiting e monitoraggio degli accessi per rilevare tentativi di autenticazione sospetti.

Infine, il sistema potrebbe essere esteso supportando meccanismi di autenticazione più avanzati, come WebAuthn o passkeys, al fine di migliorare ulteriormente sia la sicurezza sia l'esperienza utente.

7 Change Request #2: Creazione prenotazioni da parte del medico

7.1 Problema identificato

L'analisi del sistema nella versione baseline ha evidenziato una limitazione funzionale nel processo di gestione delle prenotazioni.

Nel modello iniziale del sistema, la creazione di una prenotazione poteva essere effettuata esclusivamente dal paziente tramite l'endpoint `POST /reservations`. Il medico, invece, aveva un ruolo puramente reattivo nel processo: poteva visualizzare le richieste di prenotazione e decidere se confermarle o rifiutarle, ma non poteva crearle direttamente.

Questo modello risultava limitante in diversi scenari operativi reali. In molti contesti clinici, infatti, è comune che il personale sanitario o il medico stesso inseriscano appuntamenti direttamente nel sistema durante una visita o a seguito di una comunicazione telefonica con il paziente.

L'assenza di questa funzionalità comportava quindi una rigidità nel flusso di prenotazione e non rifletteva completamente le modalità operative tipiche degli studi medici.

Di conseguenza, il sistema non consentiva ai medici di gestire in modo autonomo l'agenda delle visite, limitando l'efficienza del processo di pianificazione degli appuntamenti.

7.2 Obiettivo della modifica

La presente Change Request introduce la possibilità per il medico di creare direttamente prenotazioni per conto dei propri pazienti.

L'obiettivo principale è estendere il sistema di prenotazione introducendo un flusso alternativo rispetto a quello del paziente, mantenendo comunque le stesse regole di validazione sugli slot disponibili e sulla durata delle visite.

In particolare, la modifica ha lo scopo di:

- Consentire ai medici di creare prenotazioni direttamente per i propri pazienti.
- Differenziare il comportamento del sistema tra prenotazioni create dal paziente e prenotazioni create dal medico.

7.3 Analisi preliminare dell'impatto

7.3.1 Impatto sui requisiti

La modifica introduce un'estensione ai requisiti funzionali relativi alla gestione delle prenotazioni.

In particolare, il sistema deve ora supportare due modalità di creazione delle prenotazioni:

- creazione della prenotazione da parte del paziente;
- creazione della prenotazione da parte del medico per conto del paziente.

Dal punto di vista dei requisiti di sicurezza, il sistema deve garantire che un medico possa creare prenotazioni esclusivamente per i propri pazienti.

Inoltre, devono rimanere invariati i vincoli già presenti nel sistema, come:

- verifica della disponibilità degli slot;
- rispetto della durata delle visite;
- controllo del tipo di visita (prima visita o controllo).

Non vengono invece introdotte modifiche ai requisiti relativi alla gestione delle sessioni o all'autenticazione.

7.4 Impatto sulla struttura del codice

7.4.1 Starting Impact Set (SIS)

In fase di analisi preliminare, si è previsto che l'introduzione della funzionalità avrebbe coinvolto principalmente i componenti relativi alla gestione delle prenotazioni.

Backend

- Modifica del controller delle prenotazioni per introdurre un nuovo endpoint dedicato alla creazione di prenotazioni da parte del medico.
- Estensione del service di prenotazione per supportare la creazione di prenotazioni con stato iniziale differente.
- Introduzione di un nuovo DTO per rappresentare i dati necessari alla creazione di una prenotazione per conto di un paziente.

Frontend

- Aggiornamento dell'interfaccia utente per consentire al medico di selezionare un paziente e creare una prenotazione.
- Introduzione di nuove chiamate API per comunicare con il nuovo endpoint backend.

Database

- Nessuna modifica prevista allo schema del database, poiché la funzionalità utilizza le stesse entità già esistenti (**Reservation**, **Patient**, **Doctor**).

7.4.2 Actual Impact Set (AIS)

L'analisi del codice effettivamente modificato ha evidenziato un impatto più ampio rispetto a quanto inizialmente previsto.

Backend

- Introduzione di un nuovo DTO **CreateDoctorReservationDto** contenente il campo **patientId**.
- Refactoring del **ReservationService** tramite l'introduzione del metodo condiviso **createReservationCore**, utilizzato sia dal flusso del paziente sia da quello del medico.
- Introduzione del metodo **createReservationByDoctor** per gestire la creazione della prenotazione da parte del medico.
- Modifica del controller delle prenotazioni con l'introduzione dell'endpoint **POST /reservations/for-patient**.
- Estensione degli endpoint esistenti **/reservations/slots** e **/reservations/isFirstVisit** per supportare sia utenti pazienti sia utenti medici.
- Aggiornamento del modulo delle prenotazioni per includere nuove dipendenze (**Patient** e **Invite**) necessarie alla gestione delle query.

Frontend

- Creazione di un nuovo file API dedicato alle prenotazioni (**api/reservations.ts**).
- Introduzione di nuovi hook per la gestione delle prenotazioni e della selezione dei pazienti.
- Introduzione di componenti UI dedicati alla creazione delle prenotazioni da parte del medico, tra cui **AddMedicalReservationForm** e **PatientSelect**.

- Implementazione di un sistema di ricerca dei pazienti con paginazione infinita tramite `useInfiniteQuery`.

7.4.3 Differenza tra SIS e AIS

Backend

- È stato necessario effettuare un refactoring della logica di creazione delle prenotazioni introducendo il metodo condiviso `createReservationCore`, non previsto inizialmente.
- Gli endpoint esistenti relativi alla gestione degli slot e della prima visita sono stati estesi per supportare anche il ruolo del medico.

Frontend

- La gestione dell'interfaccia ha richiesto l'introduzione di componenti e hook dedicati, aumentando la complessità del layer frontend rispetto alle previsioni iniziali.

7.5 Stakeholder coinvolti

Gli impatti sugli stakeholder riguardano principalmente i medici e i pazienti che utilizzano il sistema per la gestione delle visite.

- **Doctor:** la modifica introduce una nuova capacità operativa per i medici, consentendo loro di gestire direttamente le prenotazioni dei pazienti e organizzare l'agenda delle visite in modo più flessibile.
- **Patient:** i pazienti beneficiano indirettamente della modifica, poiché le prenotazioni possono essere inserite direttamente dal medico anche in assenza di un'azione diretta da parte loro.

7.6 Strategia di Implementazione

La soluzione implementata introduce un nuovo flusso di creazione delle prenotazioni, distinguendo due macro-scenari operativi:

1. **Prenotazione creata dal paziente**, tramite il flusso già esistente del sistema.

2. **Prenotazione creata dal medico per conto del paziente**, tramite un endpoint dedicato che consente al medico di selezionare il paziente e creare direttamente l'appuntamento.

Nel secondo scenario, la prenotazione viene inserita direttamente nello stato **CONFIRMED**, mentre nel flusso standard creato dal paziente la prenotazione mantiene lo stato iniziale **PENDING**.

Questa distinzione consente al sistema di supportare entrambi i modelli operativi mantenendo invariata la logica di validazione relativa agli slot disponibili e alla durata delle visite.

7.6.1 Piano di implementazione

Il piano di implementazione è stato organizzato in fasi sequenziali, ciascuna delle quali produce un incremento funzionale verificabile e riduce il rischio di regressioni nel sistema esistente.

Fase 1 — Analisi del Modello di Prenotazione Esistente

La prima fase ha riguardato l'analisi del flusso di prenotazione già presente nel sistema.

In particolare sono stati analizzati:

- il controller delle prenotazioni e gli endpoint esistenti per la creazione delle visite;
- il `ReservationService`, responsabile della logica applicativa di validazione degli slot;
- le entità del dominio coinvolte (`Reservation`, `Doctor`, `Patient`);
- il comportamento dello stato iniziale delle prenotazioni create dai pazienti.

L'obiettivo di questa fase è stato individuare le parti riutilizzabili della logica esistente, evitando duplicazioni di codice e mantenendo coerenza con il modello applicativo già presente.

Risultato atteso: comprensione completa del flusso di prenotazione esistente e identificazione dei punti di estensione necessari.

Fase 2 — Implementazione Logica Backend

La seconda fase ha riguardato l'estensione della logica applicativa nel backend.

In particolare sono state introdotte le seguenti modifiche:

- introduzione di un nuovo metodo nel `ReservationService` per gestire la creazione di prenotazioni da parte del medico;

- introduzione di un metodo condiviso (`createReservationCore`) che contiene la logica comune di creazione della prenotazione;
- implementazione del metodo `createReservationByDoctor`, che utilizza la logica condivisa ma assegna automaticamente lo stato iniziale `CONFIRMED`.

Il refactoring tramite metodo condiviso consente di mantenere un'unica implementazione delle regole di validazione relative alla disponibilità degli slot e alla durata delle visite.

Risultato atteso: backend in grado di gestire due flussi distinti di creazione delle prenotazioni mantenendo un'unica logica di validazione.

Fase 3 — Estensione API e Contratti

Successivamente è stata estesa l'interfaccia API del sistema introducendo nuovi endpoint dedicati alla creazione delle prenotazioni da parte del medico.

In particolare è stato introdotto il seguente endpoint:

- `POST /reservations/for-patient`

Questo endpoint accetta un nuovo DTO contenente, oltre alle informazioni sulla prenotazione, anche l'identificatore del paziente per conto del quale viene creato l'appuntamento.

Inoltre, alcuni endpoint già esistenti sono stati estesi per supportare il nuovo flusso operativo:

- endpoint per il recupero degli slot disponibili;
- endpoint per la verifica della prima visita.

Risultato atteso: API coerenti con il nuovo modello di prenotazione e utilizzabili sia dal flusso paziente sia dal flusso medico.

Fase 4 — Integrazione Frontend

L'ultima fase ha riguardato l'integrazione della nuova funzionalità nell'interfaccia utente.

In particolare sono stati introdotti nuovi componenti e hook per supportare la creazione delle prenotazioni da parte del medico.

Le principali modifiche includono:

- implementazione di nuovi hook per la creazione delle prenotazioni e per la selezione dei pazienti;
- introduzione di componenti UI dedicati alla creazione delle prenotazioni da parte del medico;

- implementazione di un sistema di ricerca dei pazienti tramite paginazione infinita.

Il flusso operativo nel frontend prevede quindi:

1. selezione tasto per aggiungere prenotazione all'interno del calendario 7.1

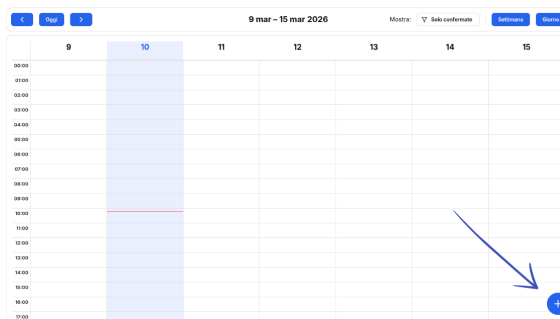


Figure 7.1: Calendario delle prenotazioni

2. selezione del paziente da parte del medico 7.2;

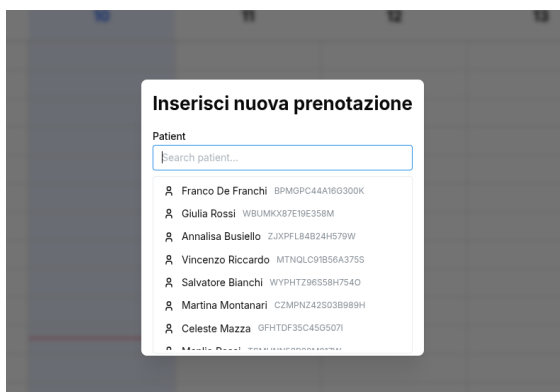


Figure 7.2: Componente selezione paziente

3. scelta della data e dello slot disponibile 7.3;

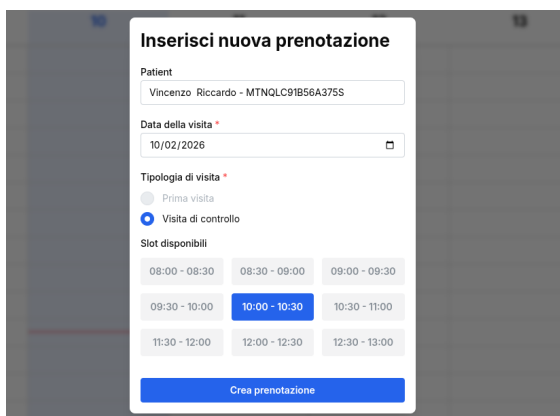


Figure 7.3: Componente selezione data e slot disponibili.

4. invio della richiesta di creazione della prenotazione tramite il nuovo endpoint backend.

Risultato atteso: integrazione completa della nuova funzionalità nel sistema, con un'interfaccia utente coerente con il resto dell'applicazione.

7.7 Tecnologie e strumenti

7.7.1 Backend

- **DTO validation** tramite class-validator per la validazione dei dati di input.

7.7.2 Frontend

- **TanStack Query** per la gestione delle chiamate API asincrone.
- componenti UI dedicati per la selezione dei pazienti e la creazione delle prenotazioni.

7.8 Gestione delle dipendenze e compatibilità

7.8.1 Dipendenze interne

La Change Request introduce nuove dipendenze tra diversi componenti del sistema.

In particolare:

- il **ReservationController** dipende dal nuovo metodo del **ReservationService**;
- il **ReservationService** utilizza repository aggiuntivi per recuperare informazioni sui pazienti;
- il frontend dipende dal nuovo endpoint API per la creazione delle prenotazioni da parte del medico.

7.8.2 Retro-compatibilità

La modifica è stata progettata per mantenere la piena retro-compatibilità con il flusso di prenotazione già esistente.

In particolare:

- il flusso di prenotazione creato dal paziente rimane invariato;
- gli endpoint esistenti continuano a funzionare senza modifiche nel contratto API;

- non sono state introdotte modifiche allo schema del database.

Questo approccio consente di introdurre la nuova funzionalità senza impattare negativamente sulle componenti già esistenti del sistema.

7.9 Aggiornamento della documentazione

L'introduzione del nuovo flusso di prenotazione richiede l'aggiornamento della documentazione tecnica del sistema.

7.10 Use Case

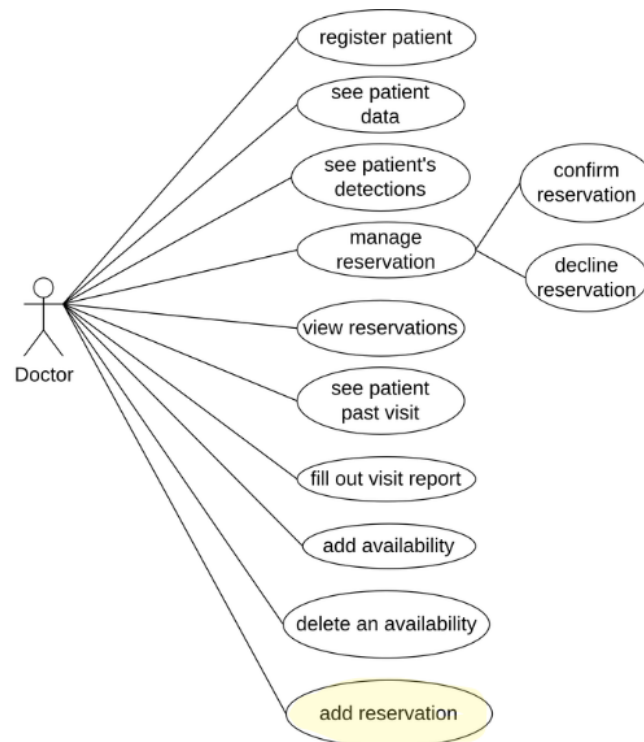


Figure 7.4: Use Case

Come mostrato in figura 7.4 è stato aggiunto un nuovo use case che rappresenterà la possibilità nel creare prenotazioni dal parte del medico.

7.11 Sequence Diagram

In particolare, è stato aggiornato il diagramma di sequenza relativo alla creazione delle prenotazioni, includendo il nuovo scenario in cui il medico crea direttamente una prenotazione per conto del paziente .

- Sequence Diagram - Medico aggiunge prenotazione

7.12 Conclusioni

L'introduzione della possibilità per i medici di creare prenotazioni per conto dei pazienti rappresenta un miglioramento significativo della flessibilità del sistema.

La modifica estende il modello esistente senza alterare la logica principale di gestione delle prenotazioni, riutilizzando gran parte delle validazioni già implementate.

Dal punto di vista architetturale, l'intervento ha comportato modifiche principalmente nel backend e nel frontend applicativo, mantenendo invariato lo schema del database.

Nel complesso, la Change Request migliora l'allineamento del sistema con i flussi operativi reali degli studi medici, aumentando l'efficienza della gestione degli appuntamenti.

8 Change Request #3: Messa in sicurezza dell'onboarding dei pazienti

8.1 Problema identificato

L'analisi della versione baseline del sistema ha evidenziato una criticità significativa nel flusso di onboarding dei pazienti.

Nel modello originario, quando un medico inseriva un nuovo paziente, il sistema creava un record nella tabella **patient** e un ulteriore record nella tabella **invite**, contenente dati anagrafici temporanei. Da tale invito veniva generato un QR Code o un link contenente il solo **inviteId**, che il paziente poteva successivamente utilizzare per completare la registrazione attraverso endpoint pubblici dedicati.

Questo approccio presentava un problema rilevante dal punto di vista della sicurezza. In particolare, il possesso del solo identificativo dell'invito risultava sufficiente per accedere al flusso di registrazione, senza ulteriori meccanismi di autenticazione o verifica dell'identità. Ne derivava la possibilità che un soggetto non autorizzato, qualora fosse riuscito a ottenere o indovinare l'**inviteId**, potesse completare la registrazione al posto del paziente legittimo.

Oltre alla criticità di sicurezza, il modello comportava anche un problema strutturale di ridondanza dei dati. Le informazioni del paziente risultavano infatti distribuite tra l'entità **Invite** e l'entità **User**, imponendo logiche di fallback in diversi punti dell'applicazione per distinguere tra pazienti già registrati e pazienti ancora associati al solo invito.

Pertanto, la criticità identificata riguarda sia la debolezza del meccanismo di onboarding, sia la presenza di una struttura dati più complessa e fragile del necessario.

8.2 Obiettivo della modifica

La presente Change Request ha l'obiettivo di eliminare il flusso di onboarding basato su inviti accessibili tramite solo identificativo e di sostituirlo con un processo di registrazione iniziale interamente gestito lato server.

L'idea alla base della modifica consiste nel creare direttamente, nel momento in cui il medico inserisce il paziente, sia il record **patient** sia il corrispondente record **user**. Il sistema genera quindi una password temporanea casuale, la salva in forma hashata e la invia automaticamente al paziente via email.

L'obiettivo specifico è:

- eliminare il rischio derivante dall'utilizzo di endpoint pubblici basati su `inviteId`;
- creare immediatamente un account utente associato al paziente al momento dell'inserimento da parte del medico;
- introdurre un meccanismo di primo accesso sicuro, imponendo il cambio password iniziale tramite un flag dedicato.

In questo modo, il sistema sostituisce un modello di attivazione differita dell'account con un modello di provisioning sicuro e controllato interamente lato backend.

8.3 Analisi preliminare dell'impatto

8.3.1 Impatto sui requisiti

La modifica proposta ha un impatto diretto sia sui requisiti di sicurezza sia sui requisiti funzionali relativi al processo di onboarding dei pazienti.

Dal punto di vista della sicurezza, il sistema deve ora garantire che la creazione dell'account utente avvenga esclusivamente a seguito di un'operazione autenticata e autorizzata da parte del medico. Inoltre, il primo accesso del paziente deve essere soggetto all'obbligo di cambio password, così da evitare l'uso prolungato di credenziali temporanee generate automaticamente.

Dal punto di vista funzionale, il requisito di registrazione del paziente viene ridefinito: il paziente non completa più autonomamente una procedura di attivazione tramite invito, ma riceve direttamente dal sistema le credenziali di accesso. Ne consegue anche una semplificazione del modello utente, poiché ogni paziente inserito dal medico deve avere fin da subito un corrispondente account applicativo.

La modifica incide inoltre sui requisiti di consistenza dei dati, rendendo necessario garantire l'univocità delle informazioni identificative, come email, telefono e codice fiscale, e prevenire la creazione di duplicati.

8.4 Impatto sulla struttura del codice

8.4.1 Starting Impact Set (SIS)

In fase preliminare, l'insieme di impatto previsto per la messa in sicurezza dell'onboarding includeva principalmente:

Backend

- modifica del flusso di creazione del paziente per includere anche la creazione del relativo account utente;
- generazione automatica di una password temporanea e relativo hashing;
- invio delle credenziali iniziali tramite email;
- introduzione di un flag per forzare il cambio password al primo accesso;
- deprecazione o rimozione degli endpoint basati su `inviteId`.

Frontend

- aggiornamento della schermata di creazione paziente per riflettere il nuovo flusso;
- rimozione della UI dedicata alla generazione e gestione del QR Code.

Database

- revisione del ruolo della tabella `invite`;
- eventuale introduzione di un attributo come `mustChangePassword`;
- possibile rafforzamento dei vincoli di unicità sui dati utente.

8.4.2 Actual Impact Set (AIS)

L'insieme di impatto effettivo è risultato più ampio e ha coinvolto una ristrutturazione più significativa del previsto.

Backend

- completa eliminazione del modulo `Invite`, inclusi entity, controller, service, repository, DTO e test dedicati;
- rimozione della registrazione di `InviteModule` dall'`AppModule`;
- rimozione degli endpoint pubblici `GET /invite/:id` e `POST /invite/:id/accept`;
- introduzione del nuovo endpoint protetto `POST /doctor/invite`, accessibile solo a utenti con ruolo `DOCTOR`;
- introduzione di un nuovo `CreateInviteDto` nel modulo `doctor`, contenente tutti i dati anagrafici e medici necessari alla creazione completa del paziente;
- estensione del `DoctorService` con il metodo `createInvite` e con diversi helper privati per:

- rilevazione duplicati;
 - creazione dell'entità `Patient`;
 - generazione e hashing della password temporanea;
 - creazione dell'entità `User`;
 - associazione tra `Patient` e `User`;
 - invio email delle credenziali.
- semplificazione di diversi metodi del `DoctorService` e del `ReservationService` tramite rimozione delle logiche di fallback precedentemente basate sull'entità `Invite`. In particolare, il metodo `getPatientById` (utilizzato dal frontend tramite la funzione `fetchPatient`) è stato rifattorizzato.

Nella versione precedente del sistema il metodo distingueva due casi:

- paziente già registrato con un'entità `User` associata;
- paziente non ancora registrato, per il quale i dati venivano recuperati dalla tabella `Invite`.

Questo comportava una query aggiuntiva verso `inviteRepository` e la costruzione dinamica della risposta utilizzando i dati dell'invito.

Con la nuova strategia di onboarding sicuro, ogni paziente viene creato direttamente con un account utente associato. Di conseguenza, il metodo è stato semplificato eliminando completamente:

- la query verso `InviteRepository`;
- il campo `inviteId` restituito al frontend;
- la logica di fallback basata sugli inviti.

Il metodo ora recupera semplicemente il paziente con la relazione `user` e restituisce una versione sicura dell'utente, priva del campo `password`.

Questo refactoring ha ridotto la complessità del metodo eliminando un ramo decisionale e una dipendenza da un modulo ormai deprecato, migliorando la manutenibilità del codice.

Frontend

- aggiornamento dell'endpoint chiamato dal frontend, da `/invite/create` a `/doctor/invite`;
- rimozione dei componenti, hook e logiche UI relativi al QR Code e all'`inviteId`;
- aggiornamento della pagina paziente e della pagina di inserimento paziente per adattarsi al nuovo flusso;
- rimozione del campo `inviteId` dalle interfacce frontend.

Database

- eliminazione dell'utilizzo operativo della tabella `invite`;
- introduzione del nuovo attributo `mustChangePassword` nell'entità `User`;
- rafforzamento della logica applicativa di controllo duplicati tramite verifica su email, telefono e codice fiscale.

8.4.3 Differenza tra SIS e AIS

Backend

- rispetto alle previsioni iniziali, non ci si è limitati a mettere in sicurezza il flusso basato su invito, ma si è proceduto alla rimozione completa dell'intero modulo `Invite`;
- la modifica ha richiesto una rifattorizzazione diffusa del `DoctorService` e del `ReservationService`, poiché numerosi punti del codice dipendevano ancora dalla presenza dell'entità `Invite`.

Frontend

- oltre al semplice aggiornamento della schermata di creazione del paziente, è stato necessario rimuovere completamente la UI associata alla condivisione del QR Code e agli identificativi di invito.

Database

- l'impatto sul modello dati è stato più rilevante del previsto, poiché la tabella `invite` è stata di fatto rimossa dal flusso applicativo e il ruolo dell'entità `User` è stato anticipato al momento della creazione del paziente.

8.5 Stakeholder coinvolti

Gli impatti sugli stakeholder riguardano principalmente i medici e i pazienti.

- **Doctor:** il medico dispone ora di un flusso più semplice e sicuro per l'inserimento di nuovi pazienti. Non è più necessario gestire QR Code o link di invito, poiché il sistema completa direttamente il provisioning dell'account e invia le credenziali al paziente.

- **Patient:** il paziente beneficia di un onboarding più lineare, ricevendo direttamente le credenziali di accesso via email. Al tempo stesso, il nuovo processo migliora la protezione del suo account, eliminando il rischio di attivazioni abusive tramite identificativi di invito esposti.

8.6 Strategia di Implementazione

La soluzione implementata distingue tre macro-scenari:

1. **Creazione sicura del paziente da parte del medico**, con generazione immediata delle entità `Patient` e `User`.
2. **Invio delle credenziali temporanee**, tramite password randomica generata lato server e inviata automaticamente via email.
3. **Primo accesso del paziente**, con enforcement del cambio password tramite flag `mustChangePassword`.

Il piano è organizzato in fasi sequenziali. Ogni fase produce un incremento testabile e riduce il rischio di regressioni.

8.6.1 Revisione del Modello Dati e del Flusso di Onboarding

La prima fase ha riguardato la revisione del modello di onboarding esistente.

In particolare:

- è stato analizzato il ruolo dell'entità `Invite` nel processo di creazione del paziente;
- è stato individuato il problema di ridondanza tra i dati temporanei dell'invito e i dati definitivi dell'utente;
- è stato introdotto il campo `mustChangePassword` nell'entità `User` per gestire il primo accesso in modo sicuro.

Risultato atteso: disponibilità di un modello dati semplificato, con account utente creato immediatamente e senza dipendenze da inviti pubblici.

8.6.2 Migrazione dei dati legacy (`Invite` → `User`)

Un aspetto critico dell'implementazione della modifica ha riguardato la gestione della retro-compatibilità con i dati già presenti nel sistema.

Nella versione precedente dell'applicazione, il flusso di onboarding dei pazienti era basato sull'entità `Invite`. Di conseguenza, nel database erano presenti numerosi record nella tabella `invite` associati a pazienti già creati ma non ancora collegati a un account utente.

La rimozione diretta della tabella `invite` avrebbe comportato la perdita di tali informazioni e reso impossibile per questi pazienti completare il processo di registrazione.

Per questo motivo è stata implementata una procedura di migrazione dei dati legacy, con l'obiettivo di trasformare ogni invito esistente in un vero e proprio account utente.

La procedura di migrazione è stata realizzata tramite uno script applicativo eseguito direttamente sul backend, che ha iterato sui record presenti nella tabella `invite` e ha effettuato le seguenti operazioni per ciascun elemento:

- recupero dell'invito e identificazione del paziente associato;
- verifica dell'assenza di un account utente già collegato al paziente;
- controllo dell'unicità di email, telefono e codice fiscale rispetto agli utenti esistenti;
- generazione di una password temporanea casuale;
- creazione del nuovo record `User` con ruolo `PATIENT`;
- impostazione del flag `mustChangePassword = true`;
- associazione del nuovo utente al paziente esistente;
- invio delle credenziali temporanee via email al paziente.

Script di migrazione `Invite` → `User`

```
1 for each invite in invite_table:
2     patient = findPatient(invite.patientId)
3
4     if patient.user exists:
5         skip
6
7     if user with same email/cf/phone exists:
8         skip
9
10    password = generateRandomPassword()
11    hashedPassword = hash(password)
12
13    newUser = createUser(invite data)
14    newUser.mustChangePassword = true
15
16    link patient -> newUser
17
18    sendEmail(invite.email, password)
```

Lo script è stato progettato per essere eseguito in modo controllato, supportando anche una modalità *dry-run* che permette di simulare la migrazione senza effettuare modifiche persistenti al database.

Questo approccio ha consentito di verificare in anticipo eventuali problemi di consistenza dei dati (come duplicati o record incompleti) prima dell'esecuzione definitiva della migrazione.

Al termine della procedura, tutti i pazienti precedentemente associati a un invito risultano dotati di un account utente valido, rendendo possibile la rimozione del modulo **Invite** senza perdita di informazioni o regressioni funzionali.

Risultato atteso: conversione completa dei record legacy basati su invito in account utente effettivi, garantendo la continuità operativa del sistema.

8.6.3 Implementazione della Logica di Onboarding nel Backend

La seconda fase ha riguardato l'implementazione della nuova logica nel backend.

In particolare:

- la logica di onboarding è stata incorporata nel modulo **Doctor**;
- è stato introdotto un nuovo endpoint protetto **POST /doctor/invite**;
- il **DoctorService** è stato esteso con un metodo pubblico di orchestrazione e con helper privati per:
 - verifica duplicati;
 - creazione dell'entità **Patient**;
 - generazione della password temporanea;
 - hashing della password;
 - creazione dell'entità **User**;
 - associazione tra paziente e utente;
 - invio email delle credenziali.

Risultato atteso: backend in grado di creare in modo atomico un nuovo paziente già provvisto di account, riducendo il rischio di inconsistenze.

8.6.4 Rimozione del Flusso Invite-Based e Hardening delle API

La terza fase ha riguardato la rimozione del flusso di onboarding basato su `inviteId`.

In particolare:

- è stato eliminato il modulo **Invite**;

- sono stati rimossi gli endpoint pubblici `GET /invite/:id` e `POST /invite/:id/accept`;
- è stata rimossa l'esclusione di tali path dal middleware di autenticazione;
- sono state eliminate le dipendenze residue da `InviteRepository` e le logiche di fallback basate sugli inviti.

Risultato atteso: eliminazione completa della superficie di attacco derivante dal possesso del solo `inviteId`.

8.6.5 Integrazione Frontend

La quarta fase ha riguardato l'adeguamento del frontend al nuovo processo.

In particolare:

- la UI di inserimento paziente è stata aggiornata per utilizzare il nuovo endpoint `/doctor/invite`;
- sono stati rimossi i componenti e gli hook dedicati alla visualizzazione del QR Code;
- è stato eliminato il campo `inviteId` dalle interfacce frontend;
- è stato semplificato il flusso operativo del medico, che ora riceve solo conferma dell'avvenuta creazione del paziente e dell'invio delle credenziali.

Risultato atteso: esperienza utente più lineare, senza meccanismi manuali di condivisione del QR Code e coerente con il nuovo onboarding sicuro.

8.7 Tecnologie e strumenti

8.7.1 Backend

- **bcrypt** per l'hashing sicuro della password temporanea.
- **crypto** per la generazione randomica della password iniziale.
- **Nodemailer** per l'invio automatico delle credenziali via email.

8.7.2 Frontend

- componenti UI esistenti adattati al nuovo endpoint e al nuovo flusso di onboarding.

8.8 Gestione delle dipendenze e compatibilità

8.8.1 Dipendenze interne

La Change Request introduce dipendenze esplicite tra:

- **DoctorController** e **DoctorService** per la creazione del paziente con account associato;
- **DoctorService** e **UserEntity** per la gestione del flag `mustChangePassword`;
- **DoctorService** e il servizio email per l'invio delle credenziali iniziali;
- **Frontend AddPatientPage** e il nuovo endpoint `/doctor/invite`.

8.8.2 Retro-compatibilità

La modifica riduce la retro-compatibilità del sistema rispetto al vecchio flusso di invito, poiché il meccanismo basato su `inviteId` viene rimosso.

Tuttavia, il comportamento generale lato utente finale rimane coerente con gli obiettivi del sistema: il medico continua a essere il soggetto responsabile dell'inserimento del paziente, ma il processo risulta ora più sicuro e più semplice.

8.8.3 Gestione del primo accesso

Il sistema crea il nuovo utente paziente con una password temporanea e con il flag `mustChangePassword` impostato a `true`. In questo modo il primo accesso può essere distinto dai successivi e sottoposto a una politica di sicurezza più restrittiva.

8.9 Aggiornamento della documentazione

8.9.1 Use Case

Sono state apportate le seguenti modifiche allo use case diagram:

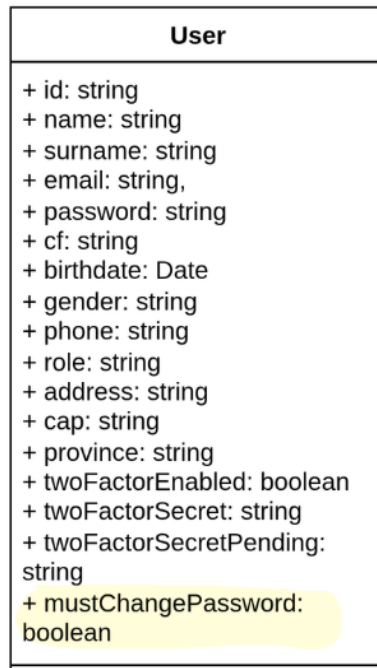
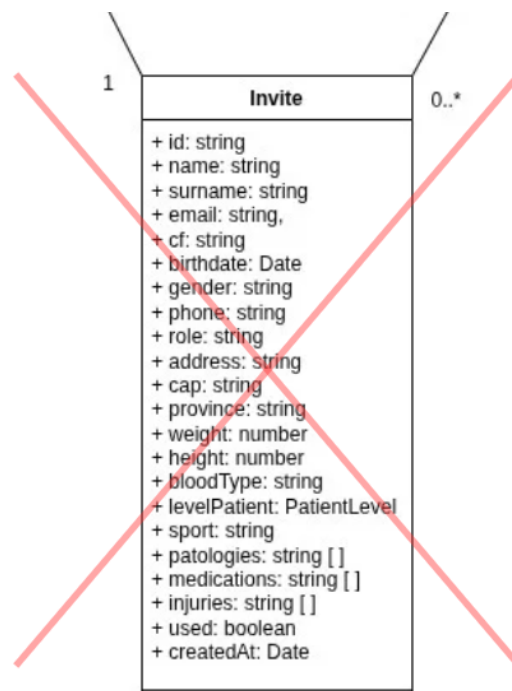
Figure 8.1: Class Diagram - Aggiunto attributo `mustChangePassword`

Figure 8.2: Class Diagram - Rimozione tabella invite

L'intero flusso viene riportato nel seguente sequence diagram, che mostra la creazione sicura del paziente da parte del medico, la generazione delle credenziali iniziali e il relativo

invio via email.

- Sequence Diagram - Secure patient onboarding

8.10 Conclusioni

La modifica ha consentito di sostituire un flusso di onboarding esposto a criticità di sicurezza con un processo più robusto, interamente gestito lato server.

L'eliminazione del modulo **Invite** ha semplificato il modello applicativo e ridotto la ridondanza dei dati, rendendo il sistema più coerente e più facile da mantenere.

L'introduzione del provisioning immediato dell'account utente e del cambio password obbligatorio al primo accesso migliora significativamente il livello di protezione del sistema senza compromettere l'operatività del medico.

8.10.1 Raccomandazioni future

Come possibile evoluzione del sistema, si suggerisce di rafforzare ulteriormente il processo di primo accesso introducendo una procedura di reset guidato della password, evitando l'invio della password temporanea in chiaro via email.

Un ulteriore miglioramento potrebbe consistere nell'introduzione di token di first-login a scadenza, utilizzabili una sola volta, così da ridurre ulteriormente i rischi associati alla gestione delle credenziali temporanee.

Infine, si potrebbe prevedere la tracciatura degli eventi di onboarding e di primo accesso in un sistema di audit logging, al fine di monitorare eventuali anomalie e migliorare la capacità di diagnosi del sistema.